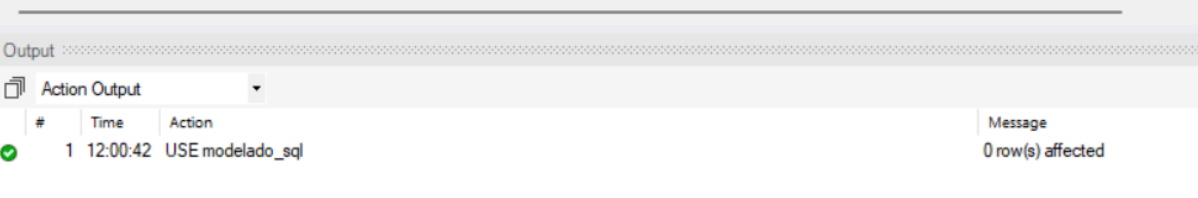


Sprint 4. Peer to peer: Ángel

NIVEL 1: Descarga los archivos CSV, estúdalos y diseña una base de datos con un esquema de estrella que contenga al menos 4 tablas de las que puedas realizar diversas consultas

- Descargamos los archivos .csv y creamos database y llamada modelado_sql. Las tablas están basadas en la info de la BBDD transactions de ejercicios anteriores, si es necesario se realizarán cambios.

```
18
19 • CREATE DATABASE modelado_sql;
20 • USE modelado_sql;
21
```



Creamos tablas european_users y tabla american_users con la info de los archivos .csv. Ángel me indica que es mejor crear la tabla conjunta y después cargar los datos, yo lo he creado de una en una y después las he unido, si mi ejercicio es incorrecto me lo comentas y lo vuelvo a hacer, es por ir adelantando trabajo.

- Info european_users (separados por ,)

```
id,name,surname,phone,email,birth_date,country,city,postal_code,address|
151,Meghan,Hayden,0800 746 6747,arcu.vel@hotmail.ca,"Jul 2, 1980",United Kingdom,London,EC1A 1BB,Ap #
432-4493 Aliquet Rd.
152,Hakeem,Alford,(0111) 367 0184,adipiscing.ligula@google.edu,"Sep 30, 1979",United Kingdom,Birmingham,B1
1AA,551-8930 Lobortis Street
153,Keegan,Pugh,(016977) 3851,sodales.nisi@aol.org,"Jul 27, 1994",United Kingdom,London,EC1A 1BB,Ap #
312-5898 Consectetur St.
154,Cooper,Bullock,(021) 2521 6627,et@outlook.net,"Nov 2, 1986",United Kingdom,Manchester,M1 1AE,872-1866
Pede Rd.
```

- Info american_users (separados por ,)

```
id,name,surname,phone,email,birth_date,country,city,postal_code,address
1,Zeus,Gamble,1-282-581-0551,interdum.enim@protonmail.edu,"Nov 17, 1985",United States,New
York,10001,348-7818 Sagittis St.
2,Garrett,Mcconnell,(718) 257-2412,integer.vitae.nibh@protonmail.org,"Aug 23, 1992",United
States,Philadelphia,19101,903 Sit Ave
3,Ciaran,Harrison,(522) 598-1365,interdum.feugiat@aol.org,"Apr 29, 1998",United
States,Houston,77001,736-2063 Tellus St.
```

```

47 • CREATE TABLE IF NOT EXISTS european_users (
48     id INT AUTO_INCREMENT PRIMARY KEY,
49     name VARCHAR(100),
50     surname VARCHAR(100),
51     phone VARCHAR(150),
52     email VARCHAR(150),
53     birth_date VARCHAR(100),
54     country VARCHAR(150),
55     city VARCHAR(150),
56     postal_code VARCHAR(100),
57     address VARCHAR(255)
58 );

```

Result Grid | Filter Rows: | Exports: | Wrap Cell Content: [IA](#)

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(100)	YES		NULL	
surname	varchar(100)	YES		NULL	
phone	varchar(150)	YES		NULL	
email	varchar(150)	YES		NULL	
birth_date	varchar(100)	YES		NULL	

Result 5 ×

Output

Action Output

#	Time	Action	Message
6	12:28:57	DESCRIBE european users	10 row(s) returned

92

```

93 • CREATE TABLE IF NOT EXISTS american_users (
94     id INT AUTO_INCREMENT PRIMARY KEY,
95     name VARCHAR(100),
96     surname VARCHAR(100),
97     phone VARCHAR(150),
98     email VARCHAR(150),
99     birth_date VARCHAR(100),
100    country VARCHAR(150),
101    city VARCHAR(150),
102    postal_code VARCHAR(100),
103    address VARCHAR(255)
104 );
105

```

Output

Action Output

#	Time	Action	Message
8	12:37:12	CREATE TABLE IF NOT EXISTS american users (id INT AUTO INCREMENT PRIMARY KEY, name...	0 row(s) affected

Cargamos con LOCAL INFILE los dos archivos , en la ruta cambiamos (\) por (/), la separación del .csv es (,)

europaean_users:

```
30
31 • LOAD DATA LOCAL INFILE "C:/ARCHIVOS/europaean_users.csv"
32 INTO TABLE europaean_users
33 FIELDS TERMINATED BY ','
34 LINES TERMINATED BY '\n'
35 IGNORE 1 ROWS
36 (id, name, surname, phone, email, birth_date, country, city, postal_code, address);
37
38
```

Output

#	Time	Action	Message
21	12:53:45	SELECT * FROM modelado sql.europaean users	3990 row(s) returned

american_users:

```
55
56 • LOAD DATA LOCAL INFILE "C:/ARCHIVOS/american_users.csv"
57 INTO TABLE american_users
58 FIELDS TERMINATED BY ','
59 LINES TERMINATED BY '\n'
60 IGNORE 1 ROWS
61 (id, name, surname, phone, email, birth_date, country, city, postal_code, address);
62
```

Output

#	Time	Action	Message
25	12:58:00	SELECT * FROM modelado sql.american users	1010 row(s) returned

Para evitar valores duplicados, CREAMOS una tabla unificada ('total_users'), con los usuarios de las dos tablas anteriores, con las mismas columnas (10) pero añadiendo una columna que muestre de cuál de las dos tablas es originario que llamaremos 'tabla_de_origen'.

```
54
55 • CREATE TABLE IF NOT EXISTS total_users (
56     id INT PRIMARY KEY,
57     name VARCHAR(100),
58     surname VARCHAR(100),
59     phone VARCHAR(150),
60     email VARCHAR(150),
61     birth_date VARCHAR(100),
62     country VARCHAR(150),
63     city VARCHAR(150),
64     postal_code VARCHAR(100),
65     address VARCHAR(255),
66     tabla_de_origen VARCHAR(10)
67 );
68
```

Output

Action Output

#	Time	Action	Message
✓ 1	09:07:09	CREATE TABLE IF NOT EXISTS total_users (id INT PRIMARY KEY, name VARCHAR(100), suma...	0 row(s) affected

Ahora INTRODUCIMOS los datos de las dos tablas, de una en una.

```
--
74 • INSERT INTO total_users
75 ( id, name, surname, phone, email, birth_date, country, city, postal_code, address, tabla_de_origen)
76 SELECT
77 id, name, surname, phone, email, birth_date, country, city, postal_code, address, 'European' AS tabla_de_origen
78 FROM european_users;
79
80 • INSERT INTO total_users
81 ( id, name, surname, phone, email, birth_date, country, city, postal_code, address, tabla_de_origen )
82 SELECT
83 id, name, surname, phone, email, birth_date, country, city, postal_code, address, 'American' AS tabla_de_origen
84 FROM american_users;
85
--
```

Output

Action Output

#	Time	Action	Message	Duration / Fe
✓ 3	09:10:11	INSERT INTO total_users (id, name, surname, phone, email, birth_date, country, city, postal_code, addr...	3990 row(s) affected Records: 3990 Duplicates: 0 Warnings: 0	0.125 sec
✓ 4	09:10:17	INSERT INTO total_users (id, name, surname, phone, email, birth_date, country, city, postal_code, addr...	1010 row(s) affected Records: 1010 Duplicates: 0 Warnings: 0	0.031 sec

Comprobamos que la tabla ha cargado todos los datos : 3990 + 1010 rows:

10

11 • **SELECT * FROM** modelado_sql.total_users;

Result Grid

	id	name	surname	phone	email	birth_date	country	city	postal_code	address	tabla_de_origen
▶	1	Zeus	Gamble	1-282-581-0551	interdum.enim@protonmail.edu	"Nov 17	1985"	United States	New York	10001	American
	2	Garrett	Mcconnell	(718) 257-2412	integer.vitae.nibh@protonmail.org	"Aug 23	1992"	United States	Philadelphia	19101	American
	3	Ciaran	Harrison	(522) 598-1365	interdum.feugiat@aol.org	"Apr 29	1998"	United States	Houston	77001	American
	4	Howard	Stafford	1-411-740-3269	ornare.egestas@idoud.edu	"Feb 18	1989"	United States	Phoenix	85001	American
	5	Hayfa	Pierce	1-554-541-2077	et.malesuada.fames@hotmail.org	"Sep 26	1998"	United States	Philadelphia	19101	American
	6	Joel	Tyson	(718) 288-8020	gravida.nunc.sed@yahoo.ca	"Oct 15	1989"	United States	San Jose	95101	American
	7	Rafael	Jimenez	(817) 689-0478	eget@outlook.ca	"Dec 4	1981"	United States	Chicago	60601	American
	8	Nissim	Franks	(692) 157-3469	egestas.aliqum.fringilla@google.ca	"Aug 1	1993"	United States	New York	10001	American
	9	Mannix	Mcdain	(590) 883-2184	aliquam.nisi@outlook.com	"Jan 24	1987"	United States	San Antonio	78201	American

total_users 4 x

Output

Action Output

#	Time	Action	Message
✓ 1	09:12:50	SELECT * FROM modelado_sql.total_users	5000 row(s) returned

Continuamos con creación y carga de tablas, con la tabla credit_cards, por seguridad añadiremos al final de la carga el nombre y orden de las columnas, ej: (id, user_id, iban, pan, pin, cvv, track1, track2, expiring_date)

91

92 • **CREATE TABLE IF NOT EXISTS** credit_cards (
93 id VARCHAR(15) PRIMARY KEY,
94 user_id INT,
95 iban VARCHAR(50),
96 pan VARCHAR(25),
97 pin VARCHAR(4),
98 cvv VARCHAR(4),
99 track1 VARCHAR(255),
100 track2 VARCHAR(255),
101 expiring_date VARCHAR(20)
102);
103

104 • **LOAD DATA LOCAL INFILE** "C:/ARCHIVOS/credit_cards.csv"
105 **INTO TABLE** credit_cards
106 **FIELDS TERMINATED BY** ','
107 **LINES TERMINATED BY** '\n'
108 **IGNORE 1 ROWS**
109 (id, user_id, iban, pan, pin, cvv, track1, track2, expiring_date);
...

Output

Action Output

#	Time	Action	Message
✓ 1	09:19:21	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(15) PRIMARY KEY, user_id INT, iban V...	0 row(s) affected
✓ 2	09:19:30	LOAD DATA LOCAL INFILE "C:/ARCHIVOS/credit_cards.csv" INTO TABLE credit_cards FIELDS TERMIN...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0

después comprobamos:

22 • **SELECT * FROM** modelado_sql.credit_cards;

Result Grid										
Filter Rows:										
Edit: Export/Import: Wrap Cell Content: Fetch rows:										
	id	user_id	iban	pan	pin	cvv	track1	track2	expiring_date	
▶	Cc5-4857	276	XX4857591835292505850771	2314242385113924	1819	467	%B2314242385113924^LWCBUDLWCBUD^22...	%B2314242385113924=2410101518363164?	09/27/25	
	Cc5-4858	277	XX8581768137002436094025	6582720299715533	3964	817	%B6582720299715533^TIQMWITIQMWI^2404...	%B6582720299715533=2411101104546272?	12/28/28	
	Cc5-4859	278	XX7826930491423553609370	8861684536289642	4983	277	%B8861684536289642^COFBDGCOFBDG^280...	%B8861684536289642=2502101761665371?	11/26/26	
	Cc5-4860	279	XX5559590368835304645299	2481155515498459	6876	661	%B2481155515498459^TIUJTUTIUJTU^31040...	%B2481155515498459=2602101514414395?	07/27/27	
	Cc5-4861	280	XX2035182877195191627307	1308930301149557	5710	398	%B1308930301149557^HPOBNZHPOBNZ^330...	%B1308930301149557=2805101751305028?	04/25/26	

credit_cards 4 x

Output

Action Output

#	Time	Action	Message
1	09:22:02	SELECT * FROM modelado_sql.credit_cards	5000 row(s) returned

Creamos y cargamos la tabla companies, comprobamos:

114	
115	• CREATE TABLE IF NOT EXISTS companies (
116	company_id VARCHAR(15) PRIMARY KEY,
117	company_name VARCHAR(225),
118	phone VARCHAR(50),
119	email VARCHAR(100),
120	country VARCHAR(100),
121	website VARCHAR(225)
122	);
123	
124	• LOAD DATA LOCAL INFILE "C:/ARCHIVOS/companies.csv"
125	INTO TABLE companies
126	FIELDS TERMINATED BY ','
127	LINES TERMINATED BY '\n'
128	IGNORE 1 ROWS
129	(company_id, company_name, phone, email, country, website);
130	

Output

Action Output

#	Time	Action	Message
1	09:25:49	CREATE TABLE IF NOT EXISTS companies (company_id VARCHAR(15) PRIMARY KEY, company_name ...	0 row(s) affected
2	09:25:56	LOAD DATA LOCAL INFILE "C:/ARCHIVOS/companies.csv" INTO TABLE companies FIELDS TERMINAT...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

14 • **SELECT * FROM** modelado_sql.companies;

Result Grid						
Filter Rows:						
Edit: Export/Import: Wrap Cell Content:						
	company_id	company_name	phone	email	country	website
▶	b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
	b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
	b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
	b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.co.uk	Germany	https://cnn.com/user/110
	b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netflix.com/settings

companies 2 x

Output

Action Output

#	Time	Action	Message
1	09:27:34	SELECT * FROM modelado_sql.companies	100 row(s) returned

Creemos y cargamos la tabla transactions, (importante, anteriormente hemos hecho incapié de la separación de los csv con una coma, en este caso se separa con punto y coma [;], necesitaremos modificar el 'FIELDS TERMINATED BY').

```

135 • CREATE TABLE IF NOT EXISTS transactions (
136     id VARCHAR(255) PRIMARY KEY,
137     card_id VARCHAR(15),
138     business_id VARCHAR(15),
139     timestamp TIMESTAMP,
140     amount DECIMAL(10, 2),
141     declined TINYINT,
142     product_ids VARCHAR(255),
143     user_id INT,
144     lat FLOAT,
145     longitude FLOAT
146 );
147 • LOAD DATA LOCAL INFILE "C:/ARCHIVOS/transactions.csv"
148 INTO TABLE transactions
149 FIELDS TERMINATED BY ';'
150 LINES TERMINATED BY '\n'
151 IGNORE 1 ROWS
152 (id, card_id, business_id, timestamp, amount, declined, product_ids, user_id, lat, longitude);
153

```

Output

Action Output

#	Time	Action	Message
5	09:49:44	LOAD DATA LOCAL INFILE "C:/ARCHIVOS/transactions.csv" INTO TABLE transactions FIELDS TERM...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0

Comprobamos.

```

13
14 • SELECT * FROM modelado_sql.transactions;

```

Result Grid

	id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
▶	00043A49-2949-494B-A5DD-A5BAE3BB19DD	CcS-9294	b-2458	2024-08-28 07:16:46	395.43	0	16, 26, 97, 87	4713	46.1999	1.43554
	000447FE-B650-4DCf-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	155.63	0	66, 69, 87	438	41.5972	12.2218
	00045D6B-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	2020-07-14 15:37:45	326.01	0	30, 11, 16, 81	2118	29.7573	-95.3796
	000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	2017-09-04 19:44:53	161.60	0	72	2115	53.5489	-113.503
	00051AA4-9CBE-4268-B070-C38062A1B3E2	CcS-7606	b-2266	2017-01-05 18:19:25	148.91	0	18	3025	52.2084	5.69081

transactions 3 x

Output

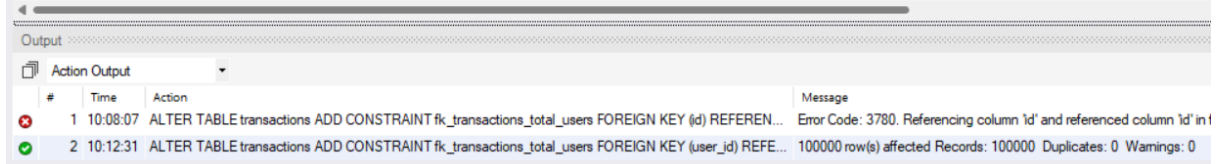
Action Output

#	Time	Action	Message
1	09:53:11	SELECT * FROM modelado_sql.transactions	100000 row(s) returned

UNIONES DE TABLAS para el Nivel 1:

- unimos transactions a total_users:

```
160
161 • ALTER TABLE transactions
162     ADD CONSTRAINT fk_transactions_total_users
163     FOREIGN KEY (user_id) REFERENCES total_users(id);
164
```

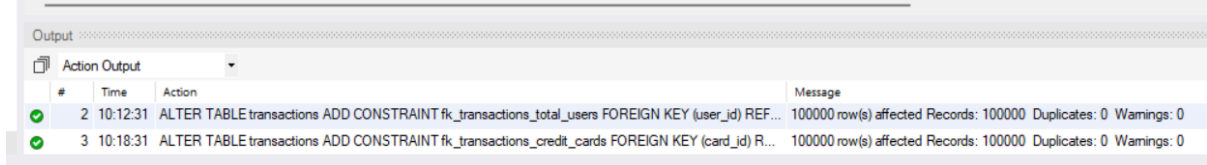


The screenshot shows the output of two SQL execution attempts. The first attempt (row 1) failed with an error: "Error Code: 3780. Referencing column 'id' and referenced column 'id' in table 'total_users' are not compatible." The second attempt (row 2) succeeded, showing "100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0".

#	Time	Action	Message
1	10:08:07	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_total_users FOREIGN KEY (id) REFEREN...	Error Code: 3780. Referencing column 'id' and referenced column 'id' in t
2	10:12:31	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_total_users FOREIGN KEY (user_id) REFE...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

- unimos transactions a credit_cards:

```
163 • ALTER TABLE transactions
164     ADD CONSTRAINT fk_transactions_credit_cards
165     FOREIGN KEY (card_id) REFERENCES credit_cards(id);
166
167
```



The screenshot shows the output of two SQL execution attempts. The first attempt (row 2) failed with an error: "Error Code: 3780. Referencing column 'id' and referenced column 'id' in table 'total_users' are not compatible." The second attempt (row 3) succeeded, showing "100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0".

#	Time	Action	Message
2	10:12:31	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_total_users FOREIGN KEY (user_id) REF...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
3	10:18:31	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_credit_cards FOREIGN KEY (card_id) R...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

- unimos transactions a companies:

```
170 • ALTER TABLE transactions
171     ADD CONSTRAINT fk_transactions_companies
172     FOREIGN KEY (business_id) REFERENCES companies(company_id);
173
```



The screenshot shows the output of two SQL execution attempts. The first attempt (row 4) failed with an error: "Error Code: 3780. Referencing column 'id' and referenced column 'id' in table 'total_users' are not compatible." The second attempt (row 5) succeeded, showing "100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0".

#	Time	Action	Message
4	10:33:39	ALTER TABLE credit_cards ADD CONSTRAINT fk_creditcards_total_users FOREIGN KEY (user_id) REF...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
5	10:36:14	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_companies FOREIGN KEY (business_id) ...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Hasta este momento y después de borrar las tablas `american_users` y `american_users` (no las necesitaremos), las tablas y sus relaciones tienen el siguiente esquema EER:

95

96 • `DROP TABLE american_users;`

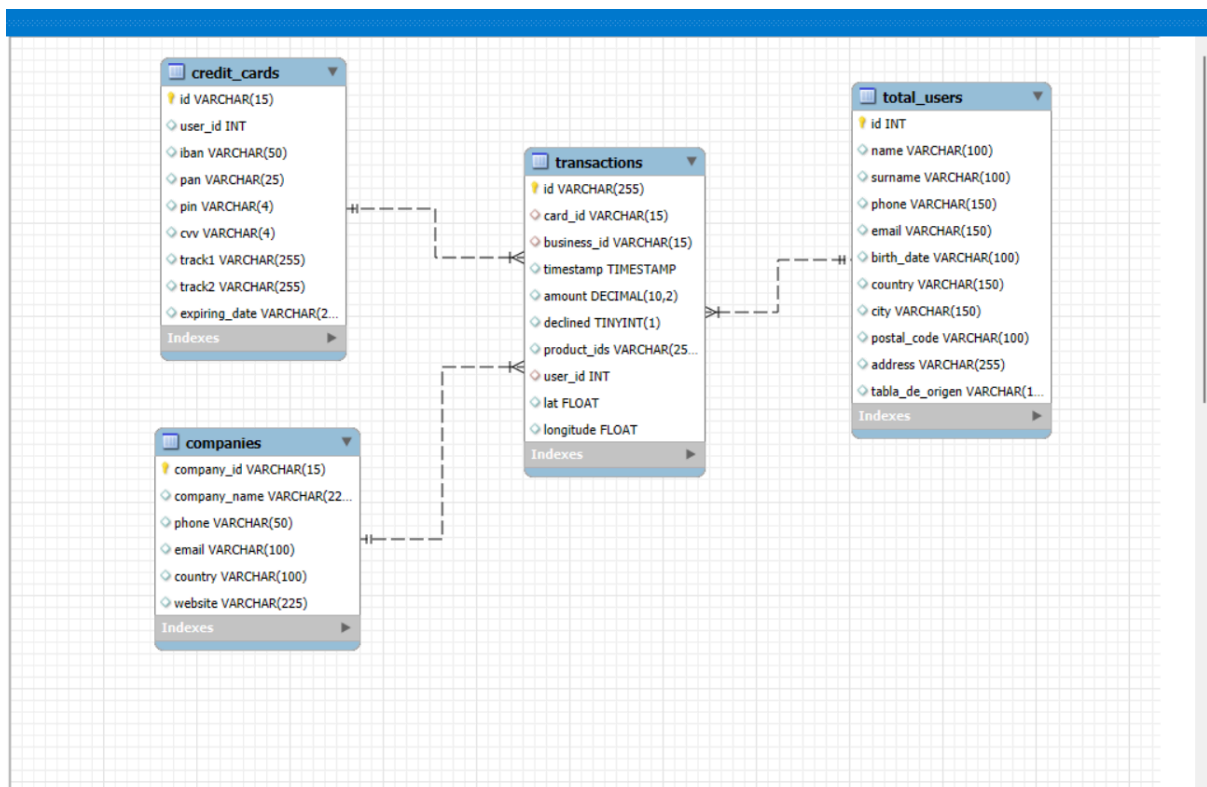
97 • `DROP TABLE european_users;`

98

Output

Action Output

	#	Time	Action	Message
✓	1	10:03:57	DROP TABLE american_users	0 row(s) affected
✓	2	10:04:01	DROP TABLE european_users	0 row(s) affected



Después de la creación y carga de tablas comenzamos con los ejercicios:

NIVEL 1, Ejercicio 1:

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 meses.

```
22
23 • SELECT id, name
24 FROM total_users
25 WHERE id IN (
26     SELECT user_id
27     FROM transactions
28     GROUP BY user_id
29     HAVING COUNT(transactions.id) > 80
30 );
31
```

Result Grid

id	name
185	Molly
289	Dxwgi
318	Bnyr
454	Sfzzoh
* NULL	NULL

total_users 11 x

Output

Action Output

#	Time	Action	Message
✓ 1	13:42:10	SELECT id, name FROM total_users WHERE id IN (SELECT user_id FROM transactions GROUP BY...	4 row(s) returned

NIVEL 1, Ejercicio 2: Muestra la media de amount por IBAN de las tarjetas de crédito a la compañía Donec Ltd, utiliza al menos 2 mesas.

```
44 • SELECT ROUND(AVG(transactions.amount),2) AS media_amount, credit_cards.iban
45 FROM transactions
46 JOIN credit_cards
47 ON transactions.card_id = credit_cards.id
48 JOIN companies
49 ON transactions.business_id = companies.company_id
50 WHERE companies.company_name = 'Donec Ltd'
51 GROUP BY credit_cards.iban, companies.company_name;
```

Result Grid			
Filter Rows:			
Export:			
Wrap Cell Content:			
	media_amount	iban	
▶	356.25	XX911406401125586307586805	
	142.96	SK9446370242474562577506	
	257.37	XX776752917845952975555640	
	139.59	XX413827362289719304908990	
	240.41	XX347787246070769610780308	
	188.58	XX683758436543000004054503	
Result 14 x			
Output			
Action Output			
#	Time	Action	Message
✓ 2	13:46:57	SELECT ROUND(AVG(transactions.amount), 2) AS media_amount, credit_cards.iban FROM transactions ...	371 row(s) returned

NIVEL 2:

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declinadas entonces es inactivo, si al menos una no es rechazada entonces es activo.

-- Primero creamos la tabla estados de tarjeta:

```
197
198 • CREATE TABLE card_status (
199     card_id VARCHAR(100) PRIMARY KEY,
200     estado VARCHAR(10)
201 );
202
203
204
```

Output

#	Time	Action	Message
1	12:47:46	CREATE TABLE card_status (card_id VARCHAR(100) PRIMARY KEY, estado VARCHAR(10)	... 0 row(s) affected

- introducimos los datos con ROW_NUMBER

```
202
203 • INSERT INTO card_status (card_id, estado)
204 SELECT subconsulta.card_id,
205     CASE
206         WHEN SUM(CASE WHEN subconsulta.declined = 0 THEN 1 ELSE 0 END) >= 1
207             THEN 'activo'
208             ELSE 'inactivo'
209         END AS estado
210 FROM (
211     SELECT transactions.card_id, transactions.declined,
212         ROW_NUMBER() OVER (
213             PARTITION BY transactions.card_id
214             ORDER BY transactions.timestamp DESC
215         ) AS posicion
216     FROM transactions
217 ) AS subconsulta
218 WHERE subconsulta.posicion <= 3
219 GROUP BY subconsulta.card_id;
220
```

Output

#	Time	Action	Message
1	12:47:46	CREATE TABLE card_status (card_id VARCHAR(100) PRIMARY KEY, estado VARCHAR(10)	... 0 row(s) affected
2	12:54:47	INSERT INTO card_status (card_id, estado) SELECT subconsulta.card_id, CASE WHEN SUM(CAS...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

Partiendo de esta tabla responde:

-- Ejercicio 1: ¿Cuántas tarjetas están activas?

```
222 • SELECT COUNT(*) AS tarjetas_activas
223 FROM card_status
224 WHERE estado = 'activo';
225
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

tarjetas_activas
4995

Result 8 x

Output

Action Output

#	Time	Action	Message
2	12:54:47	INSERT INTO card_status (card_id, estado) SELECT subconsulta.card_id, CASE WHEN SUM(C...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
3	12:57:56	SELECT COUNT(*) AS tarjetas_activas FROM card_status WHERE estado = 'activo'	1 row(s) returned

-- NIVEL 3

-- Crea una tabla con la que podamos unir los datos del nuevo archivo products.csv con la base de datos creada, teniendo en cuenta que desde transaction tienes product_ids.

– primero creamos tabla products, cargamos y comprobamos

```
86 • CREATE TABLE products(
87     id INT PRIMARY KEY,
88     product_name VARCHAR(100),
89     price VARCHAR(50),
90     colour VARCHAR(50),
91     weight INT,
92     warehouse_id VARCHAR(50)
93 );
```

Output

Action Output

#	Time	Action	Message
1	17:30:29	USE modelado_sql	0 row(s) affected
2	18:41:26	CREATE TABLE products(id INT PRIMARY KEY, product_name VARCHAR(100), price VARCHA...	0 row(s) affected

```

241 • LOAD DATA LOCAL INFILE "C:/ARCHIVOS/products.csv"
242 INTO TABLE products
243 FIELDS TERMINATED BY ','
244 ENCLOSED BY '"'
245 LINES TERMINATED BY '\n'
246 IGNORE 1 ROWS
247 (id, product_name, price, colour, weight, warehouse_id);
248
249 • SELECT * FROM modelado_sql.products;
250

```

id	product_name	price	colour	weight	warehouse_id
1	Direwolf Stannis	\$161.11	#7c7c7c	1	WH-4
2	Tarly Stark	\$9.24	#919191	2	WH-3
3	duel tourney Lannister	\$171.13	#d8d8d8	2	WH-2
4	warden south duel	\$71.89	#111111	3	WH-1
5	skywalker ewok	\$171.22	#dbdbdb	3	WH-0

products 9 x

Output

#	Time	Action	Message
2	13:07:07	LOAD DATA LOCAL INFILE "C:/ARCHIVOS/products.csv" INTO TABLE products FIELDS TERMINATE...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0
3	13:07:11	SELECT * FROM modelado_sql.products	100 row(s) returned

-- Creación de la tabla puente entre transaction y products, (para relación N a N)

```

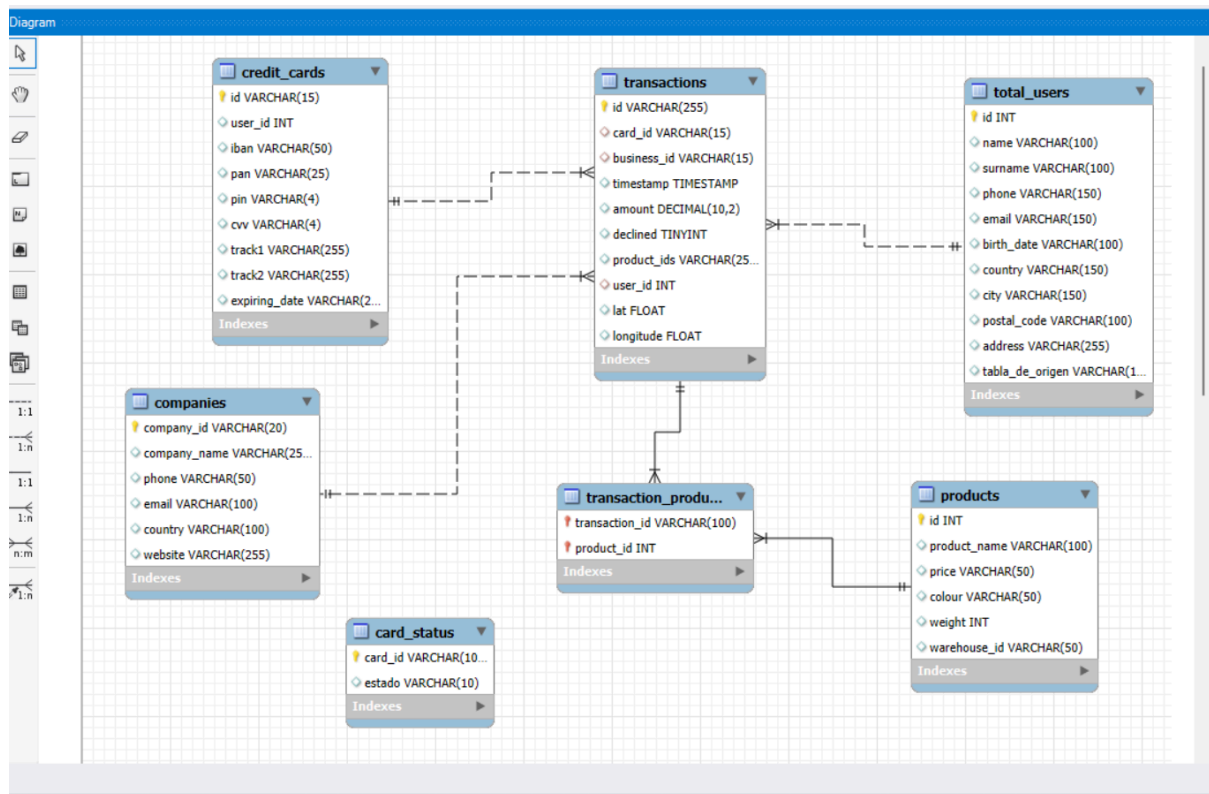
250
251 • CREATE TABLE IF NOT EXISTS transaction_products (
252     transaction_id VARCHAR(100),
253     product_id INT,
254     PRIMARY KEY (transaction_id, product_id),
255     FOREIGN KEY (transaction_id) REFERENCES transactions(id),
256     FOREIGN KEY (product_id) REFERENCES products(id)
257 );
258
259

```

Output

#	Time	Action	Message
3	13:07:11	SELECT * FROM modelado_sql.products	100 row(s) returned
4	13:09:26	CREATE TABLE IF NOT EXISTS transaction_products (transaction_id VARCHAR(100), product_id ...	0 row(s) affected

– Después de los cambios vemos las relaciones de las tablas en este EER



- NIVEL 3
- Crea una tabla con la que podamos unir los datos del nuevo archivo products.csv con la base de datos creada,
- teniendo en cuenta que desde transaction tienes product_ids. Genera la siguiente consulta:
- Ejercicio 1
- Necesitamos conocer el número de veces que se ha vendido cada producto.

Introducimos datos con JSON_TABLE

```

261
262 • INSERT INTO transaction_products (transaction_id, product_id)
263     SELECT      transactions.id, jt.product_id
264     FROM transactions
265     JOIN JSON_TABLE(
266         CONCAT('["', REPLACE(transactions.product_ids, ',', '","'), "']'),
267         '[$[*]]' COLUMNS (
268             product_id INT PATH '$'
269         )
270     ) AS jt;
271

```

Output

#	Time	Action	Message
3	09:35:10	USE modelado_sq	0 row(s) affected
4	09:35:23	INSERT INTO transaction_products (transaction_id, product_id) SELECT transactions.id, jt.product_id...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0

- validar si el INSERT del JASON_TABLE funciona

```

277 • SELECT *
278     FROM transaction_products
279     ORDER BY product_id;
280

```

Result Grid

transaction_id	product_id
001A60EA-DC9C-4E5A-9460-6628B100E7E1	1
0032F0BB-BBE6-4AA5-B5EE-EEAD533C0C48	1
00342381-503D-422D-85AB-F2D4FFAAD4C7	1
004C0A80-E537-46D8-BE44-343D2176DF15	1
004D1DB5-B2CB-4460-98B6-31C42CA96E5F	1

transaction_products3 x

Output

#	Time	Action	Message
5	09:38:28	SELECT * FROM transaction_products ORDER BY transaction_id	253391 row(s) returned
6	09:38:52	SELECT * FROM transaction_products ORDER BY product_id	253391 row(s) returned

- Ejercicio 1
- Necesitamos conocer el número de veces que se ha vendido cada producto.

```
280 • SELECT product_id, COUNT(product_id) AS veces_vendido
281 FROM transaction_products
282 GROUP BY product_id;
283
284
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	product_id	veces_vendido
▶	1	2468
	2	2573
	3	2528
	4	2584
	5	2548

Result 12 x

Output

Action Output

#	Time	Action	Message
✓ 1	11:17:17	SELECT product_id, COUNT(product_id) AS veces_vendido FROM transaction_products GROUP BY prod...	100 row(s) returned

Fin del sprint 4